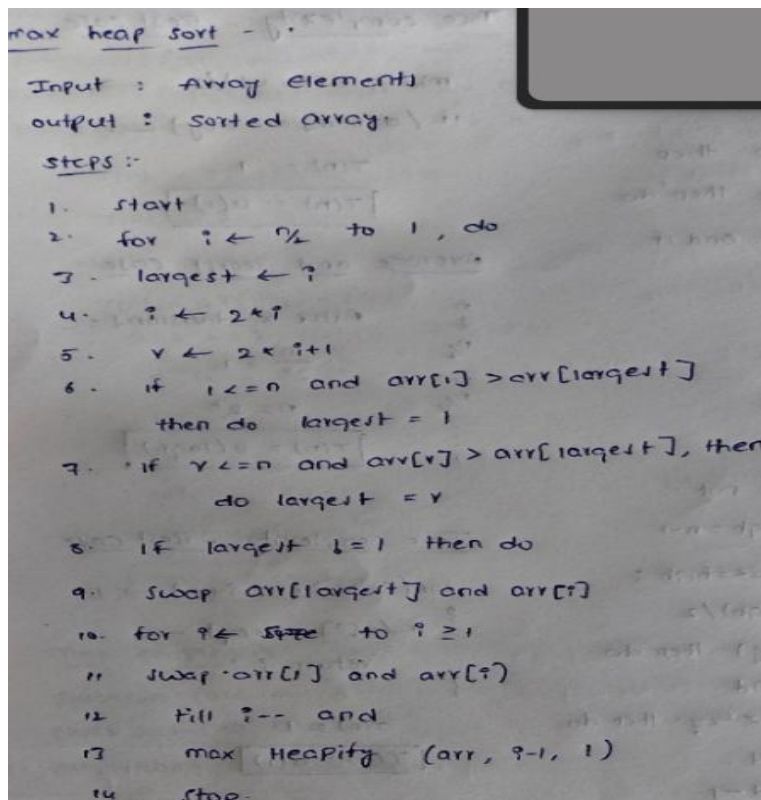




PRACTICAL - 3

AIM: Implementation of max-heap sort algorithm .

Max heap sort Algorithm :



CODE:

```
#include <iostream>
using namespace std;
void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left;
```



```
if (right < n && arr[right] > arr[largest])
    largest = right;
if (largest != i)
{
    swap(arr[i], arr[largest]);
    heapify(arr, n, largest);
}
}
void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    // Heap Sort
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
int main()
{
    int n;
    int arr[100];
    cout << "Enter size of array: ";
    cin >> n;
    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    heapSort(arr, n);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

OUTPUT:

```
Output
Enter size of array: 6
Enter array elements: 3 5 8 12 2 9
Sorted array: 2 3 5 8 9 12

=== Code Execution Successful ===
```



TIME COMPLEXITY:

Time complexity - Best, Average & worst

$$\begin{aligned} \text{for } (i = n/2 ; i > 1 ; i--) &\rightarrow n \\ \text{maxheapify}(arr, n, i) &\rightarrow \log n \\ \text{for } (i = n ; i > 1 ; i--) &\rightarrow n \\ \text{swap}(arr[i], arr[1]) &\rightarrow 1 \\ \text{maxheapify}(arr, i-1, 1) &\rightarrow \log n \\ \tau(n) &= n \times \log n * n * 1 * \log n \\ &= n \log n + n \log n \\ \tau(n) &= O(n \log n) \end{aligned}$$

CONCLUSION:

The Max-Heap Sort algorithm was successfully implemented and tested. It sorts the elements by creating a max-heap and repeatedly placing the largest element in its correct position. It is efficient for large amounts of data and has a time complexity of $O(n \log n)$.